From Inductive Definitions to a Checked Cyclic Certificate

A Theory-to-Code Tutorial for `Rocq-copy/FirstOrder`

Cyclist Rocq project documentation

Source revision
`08f6cd8834a3fc22088a064025abb9ee65c7f545`

Purpose and reader contract

This tutorial is for a reader who already knows first-order logic, inductive definitions, and advanced Rocq. Its question is not merely how the program searches, but what theorem a successful computation produces. The answer is followed from the object language through ranked semantics, theorem-backed rules, a finite cyclic graph, executable checks, a dependent certificate, and finally semantic validity.

The exposition is tied to source revision `08f6cd8834a3fc22088a064025abb9ee65c7f545`. The eleven `FirstOrder/*.v` modules and the required portions of `Cyclic/Relations.v` and `Cyclic/Framework.v` are covered. Historical plans and the implementation audit help locate design decisions, but the checked source is authoritative.

Three phrases are used precisely:

semantic validity
: a proposition quantified over every interpretation and valuation, not a Boolean test;

certified proof
: a graph and finite witness packaged with the semantic fields required by the generic soundness theorem;

bounded non-success
: a search returned `None` within a finite bound, which is neither invalidity nor a countermodel.

Every code excerpt is introduced by the mathematical object it realizes. Every operational success is connected to a checked theorem. Every known gap is stated as a completeness or interface limitation, never blurred into a soundness qualification.

Contents

Purpose and reader contract i

1 Orientation: the theorem-producing path 1

2 The mathematical object language 3

3 Ranked semantics for first-order judgments 6
3.1 Constructor-tree values and valuations . . . 6
3.2 Ranked inductive predicates . . . 7
3.3 DNF truth and shared existential witnesses . . . 8
3.4 Trace ranks . . . 9

4 The generic cyclic substrate 10
4.1 Local rules and countermodel propagation . . . 10
4.2 Finite graphs, backlinks, and structural checking . . . 11
4.3 Relations, closure, and infinite descent . . . 11
4.4 Finite witnesses and lexicographic descent . . . 12

5 The implemented inference rules I: logical structure 14
5.1 Identity modulo tags and ex falso . . . 14
5.2 Left disjunction and restricted right conjunction . . . 15
5.3 Reflexive equality elimination . . . 15

6 The implemented inference rules II: equality and structure 17
6.1 Left equality substitution . . . 17
6.2 Right existential solving and orientation . . . 17
6.3 Right discharge . . . 18
6.4 Constructor-equality decomposition . . . 18

7 The implemented inference rules III: cyclic structure 20
7.1 Explicit weakening and free substitution . . . 20
7.2 Matched and equation-producing right unfolding . . . 21
7.3 Ranked left unfolding . . . 21
7.4 Exact backlinks . . . 22

8 Verified executable infrastructure 23
8.1 Normalization and Boolean reflection . . . 23
8.2 Variables and freshness . . . 24

8.3 Directional matching . . . 24
8.4 Substitution composition and symmetric unification . . . 25

9 From syntax to candidate rules 26
9.1 Priority is operational semantics . . . 26
9.2 Scanning arbitrary positions . . . 27
9.3 Boolean side conditions are proof inputs . . . 27

10 Graph construction and certification-aware search 29
10.1 First-open-node expansion . . . 29
10.2 Prepared backlinks . . . 30
10.3 DFS, closed graphs, and checked graphs . . . 30
10.4 Trace-witness construction . . . 31

11 Reflecting executable checks into a certificate 32

12 The concrete definition library and derived mathematics 34
12.1 Nineteen inductive definitions . . . 34
12.2 Introduction, inversion, and rank induction . . . 35
12.3 Rank sums and the hydra measure . . . 36

13 Benchmarks and the public search contract 37
13.1 Nine positive judgments . . . 37
13.2 Verified proofs and iterative deepening . . . 38
13.3 Reproduced public computations . . . 39

14 Complete vertical slice: $O(x) \vdash N(x)$ 40
14.1 The direct mathematical proof . . . 40
14.2 The verified 15-node cyclic graph . . . 41
14.3 Cyclic proof and direct induction are the same descent seen twice . . . 42

15 Limitations and precise claims 44
15.1 What is implemented soundly . . . 44
15.2 Missing rule families . . . 44
15.3 Restricted implemented families . . . 45
15.4 Search and interface limitations . . . 45
15.5 Soundness is not completeness . . . 45

16 Trust boundary and reproduced validation 46
16.1 What the kernel checks . . . 46
16.2 Printed assumptions . . . 46
16.3 Build and computation record . . . 47

17 A theorem-first extension checklist 48

A Mathematics-to-Rocq correspondence 50

B Module and symbol index 52

C Rule quick reference **54**

D End-to-end audit recipe **55**

Conclusion **57**

Chapter 1

Orientation: the theorem-producing path

Mathematical intention

An automated prover should return evidence whose acceptance entails truth. The architecture studied here factors that evidence into six stages:

judgment ⟶ rule generation ⟶ proof graph ⟶ checks ⟶ certificate ⟶ semantic validity.

A judgment is finite syntax. A generator chooses only rules that already contain semantic proofs. Search arranges such instances into a finite graph. Boolean checks validate graph shape and global trace information. Reflection converts successful Booleans into dependent evidence. A well-founded descent then excludes a root countermodel. This object-logic/generic-engine split is the main architectural idea behind Cyclist [3].

Rocq realization

Figure 1.1 shows the source-level route. Solid arrows are logical or executable dependencies relevant to a theorem-producing run; dashed arrows are concrete libraries and clients. All eleven FirstOrder modules appear.

At the public boundary, `search_sound` in `FirstOrder/Search.v` is exactly the one-way contract

$$\mathsf{solvedb}(c, J) = \mathsf{true} \quad \Longrightarrow \quad \mathsf{Valid}(J). \tag{1.1}$$

The right side abbreviates $\forall w, [\![J]\!](w)$. It does not say that every valid J is found, and it does not turn `false` into a refutation.

Soundness and limitations

The trust-bearing direction is from checked evidence to semantics. Bugs in candidate order or DFS can lose proofs; they cannot manufacture a theorem unless they also pass the graph checks and the proof fields stored in each rule record. There is, however, an important operational qualification: the public strategy currently tries general graph search and then exact theorem-backed benchmark fallbacks. Thus the nine public `true` results are certified, but they are not nine demonstrations that general DFS alone rediscovered the proofs. Chapters 10 and 13 keep that distinction explicit.

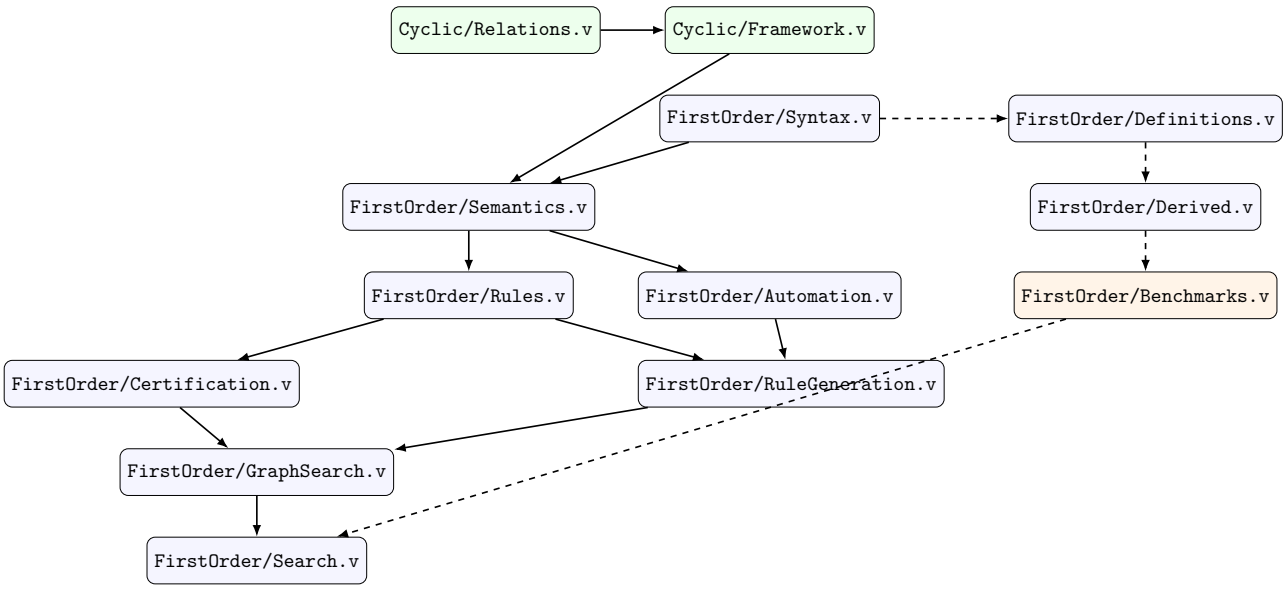


Figure 1.1: Complete module map for the documented FirstOrder implementation and the required generic Cyclic substrate.

Chapter 2

The mathematical object language

Mathematical intention

Before reading `Syntax.v`, fix two disjoint, countable classes of variables:

$$\mathcal{V} = \{\mathsf{F}_0, \mathsf{F}_1, \ldots\} \uplus \{\mathsf{E}_0, \mathsf{E}_1, \ldots\}.$$

Free variables obtain values from the world being tested. Existential variables are interpreted only on the right of a sequent, by a witness valuation. Terms form a free algebra

$$t ::= c_n \mid x \mid f(t_1, \ldots, t_k),$$

with natural-number constants and string-named function symbols. In the concrete library 0 is c_0, $s(t)$ is a unary function tree, and $\mathsf{cons}(h, t)$ is binary.

Atoms are

$$t = u, \qquad t \neq u, \qquad P^{\tau}(t_1, \ldots, t_k),$$

where $\tau \in \mathbb{N}$ is an optional trace tag. Tags affect cyclic tracing, not atom truth. A product is a conjunction $a_1 \wedge \cdots \wedge a_m$; a formula is a disjunction of products. Therefore a formula is in disjunctive normal form by construction. The empty product is truth and the empty formula is falsehood.

A sequent is $L \vdash R$, where both sides are DNF formulae. A judgment also stores an ordered definition environment. Each definition clause is a Horn-style production

$$B_1 \wedge \cdots \wedge B_m \Longrightarrow P(t_1, \ldots, t_k),$$

represented by a body product and a head term list; the predicate name comes from the environment lookup. Substitutions are finite, first-match association lists from variables to terms.

Freshness serves two independent purposes. A fresh variable offset prevents clause-local identifiers from capturing goal identifiers during unfolding. A fresh trace tag distinguishes the recursive predicate occurrences produced by a left unfold. Tag preparation supplies tags to previously untagged antecedent predicates while preserving tags already present.

Rocq realization

The core declarations in `FirstOrder/Syntax.v` closely follow those mathematical layers:

```
Inductive var_kind : Type := Free | Existential.
Record variable : Type := {
  variable_kind : var_kind;
```

```
  variable_id : nat
}.

Inductive term : Type :=
| Const (n : nat)
| Var (x : variable)
| Fun (name : string) (args : list term).

Inductive atom : Type :=
| Eq (left right : term)
| Neq (left right : term)
| Pred (tag : option nat) (name : string) (args : list term).

Definition product : Type := list atom.
Definition formula : Type := list product.
Record sequent : Type := { antecedent : formula; succedent : formula }.
Record clause : Type := { clause_body : product; clause_head : list term }.
Definition definitions : Type := list (string * list clause).
Record judgment : Type := {
  judgment_definitions : definitions;
  judgment_sequent : sequent
}.
```

Listing 2.1: The reified object language; selected declarations from Syntax.v.

The Boolean equalities are deliberately split. `atom_eqb` is exact; `atom_eqb_upto_tags` ignores only the optional tag on `Pred` and keeps equation orientation exact. This distinction is lifted through products, formulae, sequents, and `backlink_eqb` in `FirstOrder/Syntax.v`. Ordinary graph labels use `judgment_eqb`; backlink comparison may ignore tags because tags are proof annotations rather than logical content.

Substitution is a structurally recursive walk:

```
Definition substitution : Type := list (variable * term).

Fixpoint lookup_substitution (s : substitution) (x : variable)
  : option term := ...

Fixpoint subst_term (s : substitution) (t : term) : term :=
  match t with
  | Const n => Const n
  | Var x => match lookup_substitution s x with
             | Some replacement => replacement | None => Var x end
  | Fun f xs => Fun f (map (subst_term s) xs)
  end.
```

Listing 2.2: First-match substitutions and their action on terms.

The operation is extended homomorphically to atoms, products, formulae, and sequents. The syntax module also supplies `shift_term`, which preserves variable kind and increases identifiers, and `existential_shift_term`, which converts clause variables into fresh existential variables for right unfolding.

Tags are collected by `product_tags`, `formula_tags`, and `judgment_tags`. The next tag is `S(max_list(sequent_tagsq))`. The function `prepare_sequent` threads a counter through the antecedent,

attaching successive tags only to untagged predicates. It does not sort products, normalize formulas, or change the right-hand side.

Soundness and limitations

The syntax is intentionally reified and executable, but not a parser-level surface language. Constants are naturals, symbol arities are not enforced by a global signature, variables carry numeric identifiers rather than user names, and definition lookup uses the first matching string entry. Products and formulae are lists: their semantics is set-like, yet their syntax retains order and duplicates until `Automation.v` normalizes them. These choices make computation and equality straightforward, but they explain why generators must prove permutation-insensitive semantic facts and why exact backlinks are more restrictive than logical equivalence.

Chapter 3

Ranked semantics for first-order judgments

3.1 Constructor-tree values and valuations

Mathematical intention

The intended domain is the Herbrand-style free constructor algebra. Values are finite trees, so distinct constants and distinct function heads never collapse semantically. A valuation $\rho : \mathcal{V} \to \mathcal{D}$ evaluates terms homomorphically:

$$[\![c_n]\!]_\rho = \mathsf{VConst}(n), \quad [\![x]\!]_\rho = \rho(x), \quad [\![f(\vec{t})]\!]_\rho = \mathsf{VFun}(f, [\![\vec{t}]\!]_\rho).$$

This free model validates constructor clash and congruence, and it validates injectivity for every semantic `VFun`. The rule generator nevertheless uses left injectivity only for the source prover's explicit `s/cons` whitelist.

Rocq realization

```
Inductive value : Type :=
| VConst (n : nat)
| VFun (name : string) (args : list value).

Definition valuation : Type := variable -> value.

Fixpoint eval_term (rho : valuation) (t : term) : value :=
  match t with
  | Const n => VConst n
  | Var x => rho x
  | Fun f xs => VFun f (map (eval_term rho) xs)
  end.
```

Listing 3.1: The semantic carrier and term evaluation in Semantics.v.

An `interpretation` contains one function `predicate_rank:string->listvalue->optionnat`. Presence of a natural witnesses predicate truth; the number is a derivation height. Thus $P(\vec{v})$ holds exactly when $I(P, \vec{v}) = \mathsf{Some}(h)$ for some h.

Soundness and limitations

The semantics is not parametric in an arbitrary first-order structure. It fixes a free constructor domain, and equality is equality of constructor trees. This is stronger than uninterpreted first-order equality and is exactly what justifies constant/function contradiction and constructor decomposition. A future quotient or equational theory would require re-proving those rules.

3.2 Ranked inductive predicates

Mathematical intention

An interpretation models a definition environment when it supports both directions of clause reasoning:

1. *introduction*: a true clause body makes its head predicate true;
2. *ranked inversion*: every ranked head instance comes from some clause instance whose body is true, and each predicate in that body has a strictly smaller rank.

The first condition closes the relation under productions. The second rules out unsupported cycles such as defining P only from P at the same height. Equations and disequations in a body impose no rank condition because they are not inductive calls.

Rocq realization

```
Definition body_decreases
    (i : interpretation) (rho : valuation) (p : product) (height : nat) : Prop :=
  forall tag name args,
    In (Pred tag name args) p ->
    exists child_height,
      i.(predicate_rank) name (eval_terms rho args) = Some child_height /\
      child_height < height.

Definition models (ds : definitions) (i : interpretation) : Prop :=
  (forall name c rho,
      In c (lookup_definition name ds) ->
      product_holds i rho c.(clause_body) ->
      predicate_holds i name (eval_terms rho c.(clause_head))) /\
  (forall name args height,
      i.(predicate_rank) name args = Some height ->
      exists c rho,
        In c (lookup_definition name ds) /\
        eval_terms rho c.(clause_head) = args /\
        product_holds i rho c.(clause_body) /\
        body_decreases i rho c.(clause_body) height).
```

Listing 3.2: The two-sided model condition.

The concrete introduction and inversion conveniences in `FirstOrder/Derived.v`, `model_intro` and `model_elim`, are simply the two projections of this conjunction, specialized to `fo_definitions`.

Soundness and limitations

The representation packages the rank into the interpretation rather than constructing a least fixed point inside Rocq. Sound rules rely only on the two model obligations above. The semantic class is therefore precisely the class of ranked interpretations satisfying them; no unproved claim equating this record with every possible external fixed-point presentation is needed.

3.3 DNF truth and shared existential witnesses

Mathematical intention

For an interpretation I and valuation ρ, products and formulas mean

$$I, \rho \models [a_1, \ldots, a_m] \iff \bigwedge_j I, \rho \models a_j, \qquad I, \rho \models [p_1, \ldots, p_n] \iff \bigvee_k I, \rho \models p_k.$$

The empty product is true and the empty formula false. Right-hand existentials use two valuations. Define

$$(\rho \oplus \eta)(x) = \begin{cases} \rho(x), & x \text{ is free}, \\ \eta(x), & x \text{ is existential}. \end{cases}$$

Then

$$\mathsf{right_holds}(I, \rho, R) \iff \exists \eta.\ I, \rho \oplus \eta \models R.$$

The same η is used by every atom of the selected product. This is the essential point: witnesses are shared across a conjunction, but the disjunction may choose any one product.

Rocq realization

```
Fixpoint product_holds (i : interpretation) (rho : valuation)
    (p : product) : Prop :=
  match p with [] => True | a :: p' =>
    atom_holds i rho a /\ product_holds i rho p' end.

Definition formula_holds (i : interpretation) (rho : valuation)
    (f : formula) : Prop :=
  exists p, In p f /\ product_holds i rho p.

Definition merge_valuation (base witness : valuation) : valuation :=
  fun x => match x.(variable_kind) with
           | Free => base x | Existential => witness x end.

Definition right_holds (i : interpretation) (rho : valuation)
    (f : formula) : Prop :=
  exists witness, formula_holds i (merge_valuation rho witness) f.
```

Listing 3.3: Products, DNF formulae, and right-hand witnesses.

A world is a pair (I, ρ). The sequent $L \vdash R$ holds at that world when I modeling the stored definitions and L holding under ρ imply `right_holds` for R. A judgment denotes this implication, and the generic `Cyclic.Valid` quantifies it over all worlds.

Soundness and limitations

Because witnesses may differ from the base valuation on existential variables, several apparently obvious structural steps need side conditions. For example, an antecedent atom syntactically repeated on the right cannot be discharged if it contains existential variables: it may be evaluated under two different valuations. Likewise splitting a right conjunction is currently restricted to free formulae so independently obtained premise witnesses can be combined safely.

3.4 Trace ranks

Mathematical intention

A trace follows tagged inductive atoms in the antecedent. If tag t marks a true predicate whose stored derivation height is h, its semantic rank is $h + 1$. If the tagged predicate is false, its rank is zero. Missing tags also have total rank zero. The added one reserves zero as a useful false/missing sentinel and makes strict progress on a true unfolded call visible.

Rocq realization

`rank_atom` in `FirstOrder/Semantics.v` compares tags, consults `predicate_rank`, and returns `Some(Sheight)` or `Some0`. `rank_product` and `rank_formula` scan left to right; hence duplicate tags would make position matter. Left-unfold generation prevents a selected tag from appearing in the skipped prefix, while `prepare_sequent` normally gives distinct tags. `rank_of_tag` reads only the antecedent. The module `FirstOrderTheory` then instantiates the generic interface with worlds, judgments, exact and backlink equality, denotation, tag collection, and rank.

Soundness and limitations

Tags do not change truth, but they do change trace routing. Tag-insensitive backlink equality is therefore semantically sensible only because the backlink rule separately supplies a checked edge relation. Duplicate tags are not a mere cosmetic issue: rank lookup chooses the first occurrence. Generators must maintain their stated uniqueness/position side conditions.

Chapter 4

The generic cyclic substrate

4.1 Local rules and countermodel propagation

Mathematical intention

An ordinary sound rule says that validity of every premise entails validity of the conclusion. A cyclic proof needs more. If the conclusion is false at a world, the proof must be able to choose a false premise at a successor world while respecting a relation between source and target trace tags. For edge information $E = (V, P)$, the semantic obligation is

$$(a, b) \in V \Rightarrow \mathsf{rank}_b(w') \leq \mathsf{rank}_a(w), \qquad (a, b) \in P \Rightarrow \mathsf{rank}_b(w') < \mathsf{rank}_a(w).$$

Here $P \subseteq V$ is checked structurally; V gives non-increase and P strict progress.

Rocq realization

The central record in `Cyclic/Framework.v` makes both directions explicit:

```
Record rule_instance : Type := {
  rule_name : string;
  rule_conclusion : T.Judgment;
  rule_premises : list premise;
  rule_theorem : premises_valid rule_premises -> Valid rule_conclusion;
  rule_counter_step :
    forall w, ~ holds rule_conclusion w ->
    { k : nat & { p : premise & { w' : T.World |
      nth_error rule_premises k = Some p /\
      ~ holds p.(premise_judgment) w' /\
      respects_edge rule_conclusion p.(premise_judgment)
        w w' p.(premise_edge) } } }
}.
```

Listing 4.1: A theorem-backed local cyclic rule.

An `axiom_instance` is smaller: a label plus a proof of its validity. FirstOrder rules build these records directly. Search never installs a bare string such as “L.Unfold” and asks the checker to trust its meaning.

Soundness and limitations

`rule_theorem` alone would justify finite proof trees but not arbitrary cycles. `rule_counter_step` is the bridge from a hypothetical false root to an infinite false path. Conversely, a good countermodel

transition without the forward theorem would not be an acceptable user-facing inference. Both fields are part of every implemented rule.

4.2 Finite graphs, backlinks, and structural checking

Mathematical intention

A regular infinite derivation can be stored as a finite directed graph. Each node is open, closed by an axiom, expanded by a rule to fresh premise nodes, or closed by a backlink to an existing companion. A valid graph must have a root, matching labels, in-bounds edges, well-formed trace pairs, correct rule arity, no reflexive backlink, no open nodes, and no unreachable garbage.

Rocq realization

`node_kind` has constructors `OpenNode`, `AxiomNode`, `RuleNode`, and `BacklinkNode`. The immutable operations `graph_add_axiom`, `graph_apply_rule`, and `graph_add_backlink` modify only an open node after Boolean equality and bounds checks. Rule application appends fresh open nodes; backlink application requires a unary rule and uses both exact premise equality and the theory's `backlink_eqb`.

The Boolean `structural_check` in `Cyclic/Framework.v` verifies root equality, all node payloads, edge well-formedness, and reachability from index zero. Reachability is saturated for as many iterations as there are stored nodes. This rejects a syntactically closed but disconnected collection of facts.

Soundness and limitations

Graph closure is only a structural property. A cycle of identity edges can be closed and locally well formed yet fail the global progress condition. Search therefore distinguishes `graph_closedb` from a `checked_graph`; this distinction is essential rather than administrative.

4.3 Relations, closure, and infinite descent

Mathematical intention

The size-change view abstracts each proof edge to arcs between trace tags, labelled either *stay* or *decrease*. Composition connects the target tag of one arc to the source tag of the next. If several routes produce the same tag pair, decrease dominates stay. A loop is acceptable only if the transitive closure exposes a decreasing diagonal (t, t). This is the same structural idea that turns local size comparisons into a global no-infinite-run argument [4], adapted here to proof traces.

Rocq realization

`Cyclic/Relations.v` defines `slope`, `arc`, relation union and composition, bounded saturation by `relation_closure`, graph-path relations, and `relational_trace_check`. The algebra is tiny:

$$\mathsf{Decrease} \sqcup s = \mathsf{Decrease}, \qquad \mathsf{Stay} \sqcup \mathsf{Stay} = \mathsf{Stay}.$$

`path_closure` composes paths whose graph endpoints meet. `loop_relation_ok` checks decreasing diagonals only when source and target node indices agree. In the framework, `relation_fuel` and `graph_path_fuel` are finite bounds derived from graph tags and nodes.

Soundness and limitations

The executable relational check is required by `raw_check`, but the semantic certificate proof uses the more concrete finite witness described next. This is deliberate redundancy: passing the relation closure is an additional acceptance condition, not the sole semantic witness consumed by `certificate_sound`. The tutorial does not identify the implementation with every formulation of the global trace condition in the literature [1, 2].

4.4 Finite witnesses and lexicographic descent

Mathematical intention

A finite trace witness assigns each node an optional selected tag and a natural potential. Before tag selection, every followed live edge must decrease the potential. The witness may switch once from no tag to some tag. After selection, it cannot drop the tag; it follows a valid pair, and each step either has semantic progress or decreases the auxiliary potential. This turns the qualitative assertion "some trace progresses infinitely often" into a local, finite routing object.

Rocq realization

```
Record trace_witness : Type := {
  witness_tag : nat -> option nat;
  witness_potential : nat -> nat
}.

Definition trace_transition (tw : trace_witness)
    (i j : nat) (e : edge_info) : Prop :=
  match tw.(witness_tag) i, tw.(witness_tag) j with
  | None, None => tw.(witness_potential) j < tw.(witness_potential) i
  | None, Some _ => True
  | Some _, None => False
  | Some a, Some b =>
      In (a,b) e.(edge_valid) /\
      (In (a,b) e.(edge_progress) \/
       tw.(witness_potential) j < tw.(witness_potential) i)
  end.
```

Listing 4.2: Finite trace-routing data and its transition relation.

Only edges whose target is *live* are constrained. Here `livebgi` means a path of length equal to the number of graph nodes starts at i; in a finite graph such a path reaches a cycle. The complete Boolean acceptance test is

$$\mathsf{raw_check}(G, W) = \mathsf{structural_check}(G) \wedge \mathsf{relational_check}(G) \wedge \mathsf{witness_check}(G, W).$$

The certificate adds non-Boolean semantic fields: root existence and equality, false-node liveness, a false-successor/countermodel step, and propositional trace transitions. Its descent measure has two phases:

$$\mathsf{BeforeTrace}(p) \mapsto (1, (p, 0)), \qquad \mathsf{FollowingTrace}(r, p) \mapsto (0, (r, p)).$$

Nested lexicographic order over naturals is well founded. Selecting a trace decreases the leading component; later steps decrease semantic rank or, when rank merely stays, the finite potential. The

theorem `certificate_step_decreases` in `Cyclic/Framework.v` constructs a smaller false successor. `certificate_sound` in `Cyclic/Framework.v` assumes a false root and iterates that step by well-founded induction, producing a contradiction.

Soundness and limitations

The theorem uses classical double-negation elimination and the framework's constructive-choice machinery elsewhere in the development contributes to the reported assumptions of the public search theorem. These are logical-library assumptions printed by Rocq, not unchecked application axioms. Exact outputs are recorded in Chapter 16.

Chapter 5

The implemented inference rules I: logical structure

The recurring implementation pattern is worth fixing. First write a sequent rule. Then identify how a false conclusion chooses a false premise and which tag pairs are non-increasing or progressing. Only then package both proofs in a `Cyclic.rule_instance`. This order prevents an executable generator from becoming the specification.

5.1 Identity modulo tags and ex falso

Mathematical intention

Identity closes a singleton antecedent when some right product is contained in the antecedent product, comparing predicate atoms modulo tags:

$$\frac{B \subseteq_\tau A}{A \vdash R_0 \vee B \vee R_1}\ \mathrm{Id}/\tau$$

The line above is an axiom schema: the displayed condition is checked by the generator, not a proof premise. The selected product must be free so the base valuation itself supplies the right witness. Equality and disequality remain oriented and exact.

Ex falso recognizes either a reflexive disequality or a constructor clash:

$$\frac{}{A, (t \neq t), A' \vdash R}\ \bot_L \qquad \frac{}{A, (c_n = f(\vec{t})), A' \vdash R}\ \mathsf{clash}_L.$$

The symmetric function/constant orientation is also implemented.

Rocq realization

`identity_axiom_anywhere_upto_tags` in `FirstOrder/Rules.v` is named `"Id/modulo-tags"`. It accepts any RHS product position and a semantic subset proof obtained from `product_subsetb_upto_tags_spec`. `disequality_axiom` and `constructor_clash_axiom_left/right` are named `"ExFalso"`. In `RuleGeneration.v`, `identity_candidate` scans right products, while `contradiction_candidate` scans the unique antecedent product.

Soundness and limitations

The generator requires a singleton antecedent. A disjunctive antecedent must first be split. Constructor clash is broader than the old zero-versus-specific constructor test because the semantic domain makes every constant distinct from every function tree. Identity ignores predicate tags only; it does not commute equality arguments or rearrange term lists.

5.2 Left disjunction and restricted right conjunction

Mathematical intention

With DNF lists, left disjunction chooses which product makes the antecedent true. The binary decomposition is

$$\frac{p \vdash R \quad P \vdash R}{p \vee P \vdash R}\ L.\vee.$$

Repeated use enumerates every antecedent product. A single right product is a conjunction, so the implemented right rule splits its head atom from the rest:

$$\frac{L \vdash a \quad L \vdash B}{L \vdash a \wedge B}\ R.\wedge.$$

The logical rule needs compatible existential witnesses. The current kernel uses the sufficient condition that the antecedent and both right pieces are free; then witness values are irrelevant and the two premise proofs combine.

Rocq realization

`left_disjunction_rule` contains two premise records with empty trace edges, because no tagged inductive atom is being related across its split. Its countermodel proof inspects the product chosen by `formula_holds`. `right_conjunction_rule` uses identity edges and the lemmas `right_holds_free` and `product_holds_merge_free`. Generators `left_disjunction_candidate` and `right_conjunction_candidate` recognize the exact list shapes.

Soundness and limitations

The right rule is intentionally narrower than the mathematically strongest DNF conjunction rule. Independent witnesses could also be combined when the existential-variable sets of the pieces are pairwise disjoint; that condition is not implemented. Search failure on such a goal is incompleteness, not a problem with `search_sound`.

5.3 Reflexive equality elimination

Mathematical intention

Reflexive equality is semantically true, so it may be removed from either side:

$$\frac{A, A' \vdash R}{A, (t = t), A' \vdash R}\ \mathsf{Simpl{=}}L \qquad \frac{L \vdash R_0 \vee (B \wedge B') \vee R_1}{L \vdash R_0 \vee (B \wedge t = t \wedge B') \vee R_1}\ \mathsf{Simpl{=}}R.$$

Removing an equality does not alter a predicate trace rank.

Rocq realization

`reflexive_equality_rule` proves both semantic equivalence and rank equality, then invokes the reusable `equivalence_rule`. The arbitrary right-position wrapper is `right_reflexive_equality_rule`. Their generators remove exactly one selected equality; the next graph node repeats simplification if needed. The records are named `"Simpl="`.

Soundness and limitations

The implementation does not filter every reflexive equality in one graph step. This changes graph size and fuel consumption but not the logical result of repeated applications. Left scanning remains limited to a singleton antecedent, whereas right scanning visits every product and atom.

Chapter 6

The implemented inference rules II: equality and structure

6.1 Left equality substitution

Mathematical intention

If a free variable is equated to a free term, replace it throughout the remaining sequent:

$$\frac{A[x := t], A'[x := t] \vdash R[x := t]}{A, (x = t), A' \vdash R}\ \mathsf{EqSubst} \qquad \begin{array}{l} x \text{ free},\ t \text{ free}, \\ x \notin t \text{ by the checked shape used here.} \end{array}$$

At a world satisfying $x = t$, evaluating any substituted term agrees with evaluating the original. Therefore truth and trace ranks transport.

Rocq realization

`left_equality_substitution_rule` in `FirstOrder/Rules.v` stores the rule name `"Eqsubst"`. Its proof is built from structural lemmas `eval_term_single_subst_equal`, `atom_holds_single_subst_equal`, and their product/formula/rank lifts. `find_left_equality_substitution` recognizes only a free variable syntactically on the left of an equation and a free replacement.

Soundness and limitations

The reverse orientation $t = x$ is not connected on the left, even though some cases are sound. Nor is the rule a general equality solver. The narrow shape makes its valuation proof direct and avoids hidden cyclic substitutions.

6.2 Right existential solving and orientation

Mathematical intention

Inside one selected right product, an equation can assign a witness variable:

$$\frac{L \vdash R_0 \vee (B[\mathsf{E}_x := t] \wedge B'[\mathsf{E}_x := t]) \vee R_1}{L \vdash R_0 \vee (B \wedge \mathsf{E}_x = t \wedge B') \vee R_1}\ \exists\mathsf{Solve} \qquad t \text{ free}.$$

The conclusion witness is obtained from the premise witness by overriding E_x with the base interpretation of t. The symmetric equation $t = \mathsf{E}_x$ is first oriented.

Rocq realization

`single_existential_substitution` and `existential_solution_valuation` formalize that override. `right_existential_substitution_rule` is named `"Eqexistential/right"`; its arbitrary-position wrapper preserves other products. `right_equality_orientation_rule` swaps only the selected equation. The simplifier tries direct solving, then orientation, and on the next node solves the oriented result.

Soundness and limitations

This is right-side witness solving, not the missing whole-sequent *existential instantiation* rule of the original prover. It requires a free replacement and does not solve existential-to-existential cycles or arbitrary terms containing existential variables.

6.3 Right discharge

Mathematical intention

An atom already supplied by a singleton antecedent may be removed from a right product:

$$\frac{a, A \vdash R_0 \vee (B \wedge B') \vee R_1}{a, A \vdash R_0 \vee (B \wedge a \wedge B') \vee R_1}\ \mathsf{Discharge}_R.$$

Predicate comparison ignores tags, because tags do not affect truth. The discharged atom must be free so its truth under the base valuation persists under the merged right witness.

Rocq realization

`right_discharge_rule` uses `atom_eqb_upto_tags_holds` and `atom_holds_merge_free`; `right_discharge_candidate` searches every right product and position. The rule is generated before constructor equality decompositions, matching the documented simplifier priority.

Soundness and limitations

Dropping the free-atom check would be unsound: an existential variable in the antecedent is read from ρ, but the same syntax on the right is read from $\rho \oplus \eta$. The regression `generated_right_discharge_rejects_existential_atom` fixes this boundary.

6.4 Constructor-equality decomposition

Mathematical intention

On the right, congruence reduces equality of same-headed, equal-arity function terms to component equations:

$$\frac{L \vdash R_0 \vee (B \wedge \bigwedge_i s_i = t_i \wedge B') \vee R_1}{L \vdash R_0 \vee (B \wedge f(\vec{s}) = f(\vec{t}) \wedge B') \vee R_1}\ \mathsf{FunEq}_R.$$

In a free constructor algebra the converse is also valid. The left rule uses that injectivity, but only for the configured constructors s and cons:

$$\frac{A, (s_1 = t_1), \ldots, (s_k = t_k), A' \vdash R}{A, f(\vec{s}) = f(\vec{t}), A' \vdash R}\ \mathsf{Inj}_L.$$

Rocq realization

`equation_product` zips equal-length argument lists. The semantic and rank lemmas justify `right_function_equality_rule` and `left_function_equality_rule`. `injective_constructorb` accepts only the strings `"s"` and `"cons"`. Both generators scan arbitrary positions and reject arity mismatch safely.

Soundness and limitations

Right decomposition needs only congruence and therefore accepts any matching function name. Left decomposition uses injectivity and intentionally follows the source prover's whitelist, despite the local semantic domain validating more constructors. Generalizing the whitelist is sound for the current semantics but would be an interface and search-policy change, not a mere proof cleanup.

Chapter 7

The implemented inference rules III: cyclic structure

7.1 Explicit weakening and free substitution

Mathematical intention

Non-exact backlinks are prepared by ordinary proof steps rather than hidden inside the backlink checker. Prefix weakening removes extra antecedent atoms:

$$\frac{A \vdash R}{A, E \vdash R}\ \mathsf{Weak}.$$

Free substitution moves from a general companion to its instance:

$$\frac{L \vdash R}{\sigma(L) \vdash \sigma(R)}\ \mathsf{Subst}, \qquad \sigma \text{ maps only free variables to free terms.}$$

These directions are chosen so that the current open node becomes exactly the ancestor label after one or more explicit steps.

Rocq realization

`prefix_weakening_rule` is named `"Weaken"` and retains rank information for tags in the smaller prefix. `free_substitutionb` checks both domain kinds and range terms. The general rule `general_free_substitution_rule`, named `"Subst"`, constructs a new valuation by evaluating substituted terms and proves truth/rank transport. `GraphSearch.v` invokes these records while preparing backlinks.

Soundness and limitations

Weakening is prefix-only, not arbitrary set subsumption. Judgment matching flattens terms, directionally matches them, and then rechecks the entire substituted judgment exactly. This is a sound but narrow substitute for full formula/product unification and subsumption.

7.2 Matched and equation-producing right unfolding

Mathematical intention

Suppose $P(\vec{h}) \leftarrow B$ is a definition clause and a directional match $\sigma(\vec{h}) = \vec{t}$ succeeds. Right introduction replaces the selected goal predicate by the instantiated clause body:

$$\frac{L \vdash R_0 \vee (A \wedge \sigma(B) \wedge A') \vee R_1}{L \vdash R_0 \vee (A \wedge P(\vec{t}) \wedge A') \vee R_1}\ R.\mathsf{Unfold/match}.$$

When matching fails but arities agree, the fallback shifts clause variables to fresh right existential variables and emits head equations:

$$\frac{L \vdash R_0 \vee (A \wedge B^{\exists} \wedge \vec{t} = \vec{h}^{\exists} \wedge A') \vee R_1}{L \vdash R_0 \vee (A \wedge P(\vec{t}) \wedge A') \vee R_1}\ R.\mathsf{Unfold}.$$

Clause introduction proves both rules: satisfying the body under a clause valuation makes the head predicate true.

Rocq realization

`matched_right_unfold_rule` and its arbitrary-position wrapper carry a proof from `match_terms_sound`; their name is `"R.Unfold/match"`. `right_unfold_rule` and `standard_right_unfold_at_rule` implement the equation-producing fallback, named `"R.Unfold"`. The latter uses `existential_shift_product` and `equation_product`. Both preserve the left trace ranks with identity-style edges.

Soundness and limitations

Directional matching binds pattern variables only. The standalone symmetric unifier is not integrated into right unfolding, so the behavior is not the source prover's full multi-unification with residual equations. Matched unfolding is tried before left unfolding; the equation-producing family is placed after left unfolding to avoid spending DFS fuel prematurely on weak constraint branches.

7.3 Ranked left unfolding

Mathematical intention

Left unfolding is the induction-bearing rule. For a tagged antecedent atom $P^t(\vec{u})$, ranked inversion chooses one defining clause. To prove the conclusion, every possible clause must be handled:

$$\frac{\{\ A, B_c^{\uparrow, t_c}, \vec{u} = \vec{h}_c^{\uparrow}, P^t(\vec{u}), A' \vdash R\ \}_{c \in \mathsf{def}(P)}}{A, P^t(\vec{u}), A' \vdash R}\ L.\mathsf{Unfold}.$$

The schematic premise retains the selected predicate as a contraction point. Clause variables are freshly shifted. Body predicates receive a fresh tag. An edge relates the source tag to both contraction and body tags; recursive body occurrences carry strict progress because ranked inversion makes their heights smaller.

Rocq realization

`left_unfold_product`, `left_unfold_judgment`, and `left_unfold_premises` construct the branches. The proof `left_unfold_respects` uses model inversion to select the false premise and establish edge inequalities. `left_unfold_rule`, named `"L.Unfold"`, produces one premise per clause. The arbitrary-position wrapper `left_unfold_at_rule` rotates products while proving truth and rank invariance.

The generator requires a singleton antecedent, a selected tagged predicate, a fresh distinct tag, variable-bound checks, a free RHS, and no duplicate selected tag in the skipped prefix. It scans every atom of the unique product in source order.

Soundness and limitations

The free-RHS restriction means left unfolding is not connected when the right side contains existential variables. There is no dedicated composed left-unfold-then-right-unfold candidate. Both choices reduce search coverage, not rule soundness. The contraction copy is crucial: it provides the repeated shape against which a later backlink can close.

7.4 Exact backlinks

Mathematical intention

A backlink identifies a repeated sequent after ordinary preparation steps:

$$\frac{J}{J}\ \mathsf{Backlink}.$$

This rule is locally just semantic equivalence. Its cyclic force comes from where its only premise points in the finite graph and from progress accumulated along the complete loop. The backlink edge itself preserves each antecedent tag; it need not be the edge on which strict decrease occurred.

Rocq realization

`exact_backlink_rule` specializes `backlink_rule` to the same judgment with `identity_edge`. Search considers every earlier node index. `graph_add_backlink` independently checks source conclusion, target premise, tag-insensitive backlink equality, bounds, unary arity, and non-reflexivity.

Soundness and limitations

The final backlink is exact in list shape and logical syntax. Tag-insensitive comparison alone is not a general subsumption algorithm. Substitution and prefix weakening can prepare some companions, but arbitrary reordering, set-based weakening, equation orientation alternatives, and selected-tag minimization are absent. Folding before backlinking is entirely absent.

Chapter 8

Verified executable infrastructure

`FirstOrder/Automation.v` is not a bag of trusted tactics. It defines total Gallina programs, proves small reflection or preservation lemmas around their results, and lets later modules consume only those proved interfaces. This chapter follows its three main layers: set-style normalization, variable analysis, and term solving.

8.1 Normalization and Boolean reflection

Mathematical intention

Although products and formulae are stored as lists, conjunction and disjunction are idempotent and insensitive to duplicates. A safe normalization may remove duplicates without changing denotation:

$$[\![\mathsf{nodup}(p)]\!] = [\![p]\!], \qquad [\![\mathsf{nodup}(\mathsf{map}\ \mathsf{nodup}\ F)]\!] = [\![F]\!].$$

No sorting is mathematically required for this fact. Exact Boolean equality can then serve as a decidable guard, provided a reflection theorem connects `eqb=true` to propositional equality.

Rocq realization

`normalize_product` is `nodupatom_eq_dec`; `normalize_formula` maps product normalization and removes duplicate products. The chain `normalize_product_holds`, `normalize_formula_holds`, `normalize_right_holds`, `normalize_sequent_holds`, and `normalize_judgment_valid` proves preservation at every layer. This last equivalence is why a public `verified_proof` may certify the normalized root and still yield validity of the exact input.

Set-like Boolean helpers have specifications rather than being assumed: `product_set_eqb_spec` characterizes mutual membership, and `product_mem_setb_spec` characterizes membership of a product modulo atom permutation. The syntax module similarly provides `term_eqb_spec`, `atom_eqb_spec`, `product_subsetb_spec`, and the tag-insensitive subset specification.

Soundness and limitations

Normalization removes duplicates but does not canonicalize order. Two logically equivalent permutations can therefore remain syntactically unequal. Later exact checks deliberately remain conservative. Replacing `nodup` with a canonical ordering would require an ordering on all reified syntax and new preservation/equality results.

8.2 Variables and freshness

Mathematical intention

Matching and unfolding need explicit finite variable information: variables occurring in a term, whether a variable belongs to the free or existential class, and identifiers unused by an entire sequent. Freshness is numerical: one plus the maximum identifier suffices because syntax is finite.

Rocq realization

`term_variables` recursively concatenates variables from function arguments. Atom, product, formula, and sequent collectors lift it. `is_free_variable` and `is_existential_variable` inspect `variable_kind`; `fresh_free_variable` and `fresh_existential_variable` reuse `fresh_variable_offset` with the desired kind. `term_is_existential_variable` recognizes the exact single-variable term shape used by right equality solving.

Soundness and limitations

These collectors are executable analyses, not binders or a locally nameless metatheory. Capture avoidance is achieved by arithmetic shifting and checked bounds. Any extension that adds binders to terms would invalidate this simple freshness story and require a new syntax and semantic substitution theory.

8.3 Directional matching

Mathematical intention

Directional matching treats the left list as patterns and the right list as fixed targets. It may bind a variable occurring in the pattern, but it does not solve target variables symmetrically. Repeated pattern variables must receive consistent terms, constructors must have the same name and arity, and the lists must have the same length.

The robust proof pattern is *compute, then verify*:

$$\mathsf{rawMatch}(\vec{p},\vec{t}) = \mathsf{Some}(\sigma), \quad \mathsf{eqb}(\sigma\vec{p},\vec{t}) = \mathsf{true} \quad \Longrightarrow \quad \sigma\vec{p} = \vec{t}.$$

Correctness of downstream rules depends on the final equality, not on a large invariant about every branch of the heuristic algorithm.

Rocq realization

`zip_terms` creates an equation work list. `match_equations` consumes a structurally bounded amount of fuel, uses `add_match_binding` to reject inconsistent rebinding, and descends through matching function heads. `raw_match_terms` exposes that engine; `match_terms` re-applies the candidate and checks `terms_eqb`. The gate is:

```
Lemma match_terms_sound patterns targets s :
  match_terms patterns targets = Some s ->
  map (subst_term s) patterns = targets.
```

Listing 8.1: The small theorem consumed by matched unfolding.

Soundness and limitations

Matcher fuel exhaustion is non-success. Directionality is a design feature, not a completeness theorem. In particular, it does not reproduce the source prover's multi-unification behavior for repeated variables and residual equations.

8.4 Substitution composition and symmetric unification

Mathematical intention

Symmetric first-order unification may solve variables on either side. When a new binding $x \mapsto t$ is found, it must update the remaining equations and all old substitution ranges. An occurs check rejects $x \mapsto f(\ldots x \ldots)$. Composition order matters because association-list lookup uses the first binding.

Rocq realization

`compose_substitutionneweroldер` applies the newer substitution to old ranges and then appends genuinely new-domain bindings. `term_occursb` implements the structural occurs check. `unify_equations` handles equality, variables in either orientation, constant clash, and same-headed function decomposition. A generous size-based `unification_fuel` makes it total. As with matching, the exported `unify_terms` verifies the result by reapplying it to both sides, and `unify_terms_sound` exposes only the proved equality.

Two source regressions record the intended behavior:

```
Example unification_propagates_later_bindings :
  unify_terms [Var (free_var 0); Var (free_var 1)]
    [Var (free_var 1); Const 0] =
  Some [((free_var 1), Const 0); ((free_var 0), Const 0)].

Example unification_occurs_check :
  unify_terms [Var (free_var 0)]
    [succ (Var (free_var 0))] = None.
```

Listing 8.2: Executable unification boundaries.

Soundness and limitations

The unifier is verified when it succeeds, but the fuel bound is not proved complete for all unifiable inputs. More importantly, current rule generation does not use it to implement general right unfolding or backlink subsumption. It is verified infrastructure awaiting a carefully specified integration.

Chapter 9

From syntax to candidate rules

9.1 Priority is operational semantics

Mathematical intention

A rule kernel describes what an inference means. A generator describes where to apply it and in what order alternatives should be explored. The distinction is important: two generators can be equally sound but have very different coverage and runtime. Candidate order is therefore part of the executable behavior, even though it is not part of the validity theorem.

Rocq realization

`FirstOrder/RuleGeneration.v` first tries an axiom. Within `generate_axiom`, identity is tried before contradiction. The single `simplification_candidate` returns at most one rule, in this precise order for singleton antecedents:

1. left reflexive equality;
2. right reflexive equality;
3. left equality substitution;
4. right existential solving or orientation;
5. right discharge;
6. left constructor injectivity;
7. right function congruence.

With a non-singleton antecedent, left-only simplifications and discharge are unavailable; right reflexivity, existential solving, and function equality remain.

The ordinary list returned by `generate_rules` is ordered as

$$L.\vee, R.\wedge, R.\mathsf{Unfold/match},\ L.\mathsf{Unfold},\ R.\mathsf{Unfold}\text{ with equations.}$$

Every item is already a complete `rule_instance`; the generator does not emit unverified descriptors.

Soundness and limitations

This order mirrors only the implemented portion of the source prover. There is no folding family, no general existential instantiation, and no composed left/right unfold primitive. Moving a sound candidate earlier cannot make the system unsound, but it can make bounded DFS miss proofs it previously found by using the bound on a worse branch.

9.2 Scanning arbitrary positions

Mathematical intention

List representation should not restrict a set-like logical rule to the first atom. A standard technique carries a prefix accumulator B, inspects the head a of the remaining list, and reconstructs the source as $B +\!\!+ a :: A$. For nested DNF, one accumulator tracks prior products and another prior atoms. A transport lemma proves that rotating or replacing at the chosen position preserves semantics and the relevant ranks.

Rocq realization

This pattern appears in `find_right_reflexive_in_product/formula`, `find_right_existential_in_product/formula`, `find_right_discharge_in_product/formula`, both function-equality scans, and right unfolding. Left unfolding scans every atom of the unique antecedent product and uses `left_unfold_at_candidate` to establish the no-duplicate-tag side condition before constructing a wrapper record.

Soundness and limitations

Arbitrary position does not mean arbitrary logical equivalence. Prefix accumulators preserve the exact order of untouched atoms and products. This predictability makes graph labels reproducible, but it also exposes list-order limitations in later backlink matching.

9.3 Boolean side conditions are proof inputs

Mathematical intention

An executable test such as “this term is free” becomes useful to a proof only through an equation or reflection lemma. Candidate construction should destruct the Boolean and pass the successful equation into a theorem-backed constructor. Rechecking whole outputs is especially valuable when a helper flattens or approximates structured syntax.

Rocq realization

Examples include `formula_freeb=true` passed to `right_conjunction_rule`, `free_substitutionb=true` passed to `general_free_substitution_rule`, length equality passed to standard right unfold, and `match_terms_sound` passed to matched unfold. For prepared backlinks, `checked_judgment_match` first flattens all terms, runs matching, and then insists that substituting the complete structured judgment gives exact `judgment_eqb=true`. Predicate names, tags, connectives, definitions, and list shape are therefore rechecked.

Soundness and limitations

Reflection proves the accepted result, not completeness of the Boolean procedure. A false test may mean a genuine violation or simply that the current approximation did not recognize a safe case. The public contract is designed around this asymmetry.

Chapter 10

Graph construction and certification-aware search

10.1 First-open-node expansion

Mathematical intention

Search maintains a persistent finite graph and always expands its first open node. Consequently all lower indices are already closed, so a backlink may safely enumerate indices $0, \ldots, i-1$ as potential companions. A single expansion returns a list of successor graphs in priority order.

Rocq realization

`first_open_from` scans node payloads; `graph_closedb` is true when the scan returns `None`. `expand_graph` in `FirstOrder/GraphSearch.v` then builds successors in this order:

1. a generated axiom, if any;
2. the first simplification candidate, if any;
3. exact backlinks to earlier nodes;
4. prepared substitution or weakening steps toward earlier nodes;
5. ordinary candidates from `generate_rules`.

Every transition calls a framework graph operation, which rechecks label agreement and openness. The search module cannot mutate a node payload directly.

Soundness and limitations

First-open expansion fixes a useful deterministic traversal but may repeat equivalent subproblems. There is no memo table, no semantic subsumption cache, and no fair interleaving across unbounded depths. These are performance and completeness issues only because final success is rechecked.

10.2 Prepared backlinks

Mathematical intention

If an ancestor A is not exactly the current node C, search may still reach it through explicit sound rules. Two recognized cases are:

$$C = \sigma(A) \quad \rightsquigarrow \quad \mathsf{Subst}; \qquad C = A \wedge E \quad \rightsquigarrow \quad \mathsf{Weaken}.$$

After the preparation node, ordinary expansion can discover an exact backlink. Keeping preparation explicit makes both the proof graph and its trace edges auditable.

Rocq realization

`judgment_search_terms` flattens terms from both formulae. `checked_judgment_match` matches ancestor to current and rechecks the whole judgment. `substitution_rule_to_ancestor` additionally demands a nonempty, free substitution. `weakening_rule_to_ancestor` supports only a unique antecedent product, splits the current product at the ancestor length, and checks the projected prefix. The functions `prepared_backlink_successor(s)` merely insert the corresponding verified rule; they do not install a backlink yet.

Soundness and limitations

The weakening test assumes the useful atoms form a prefix. Matching is directional and exact after substitution. Thus many logically adequate companions are not recognized. Full product/formula subsumption, permutation, equation normalization, and tag minimization remain future work.

10.3 DFS, closed graphs, and checked graphs

Mathematical intention

Depth-first search with fuel is total. The crucial question is when a leaf is accepted. “No open nodes” is insufficient; a closed cyclic graph may lack a progressing trace. Certification-aware DFS must run the global checker at a closed leaf and, if that leaf fails, return to the local successor loop and try the next branch.

Rocq realization

`bounded_depth_first_search` is retained as a diagnostic returning the first structurally closed graph. The public path uses `bounded_checked_depth_first_search`. Its return type is a dependent sum

$$\mathsf{option}\ \{G : \mathsf{graph}\ \&\ \mathsf{checked_graph}(G)\}.$$

A `checked_graphg` contains a witness and the equation `Cyclic.raw_checkgwitness=true`. At an invalid closed graph the strong DFS yields `None` to `try_successors`, not to the whole search immediately.

Soundness and limitations

Fuel decreases once per whole graph-state expansion, not per logical proof height in the source OCaml sense. Branching, repeated one-atom simplification, and explicit backlink preparation therefore affect the numerical threshold. A zero-fuel call may still accept an already closed and checked graph.

10.4 Trace-witness construction

Mathematical intention

Witness search can exploit a cheap deterministic guess and still remain sound by checking it. A complete finite fallback can enumerate every optional tag available at each node and every potential in a graph-size bound. Heuristics affect only when a witness is found, never what acceptance means.

Rocq realization

`heuristic_tag_at` chooses the first syntactic tag at each live node and none at dead nodes. This aligns with left unfolding, which places a freshly retagged recursive body before the contraction copy. Potentials are computed by repeated backward relaxation: a progressing selected pair needs zero extra potential, while a non-progressing edge needs one more than its target. `find_graph_witness` short-circuits in three stages:

1. heuristic tags and relaxed potentials;
2. the empty all-none/all-zero witness, useful for acyclic graphs;
3. exhaustive tag/potential enumeration.

Every candidate is accepted only by `Cyclic.raw_check`.

Soundness and limitations

The Cartesian fallback can be expensive. Its bounded exhaustion is again non-success. The heuristic's choice of the first tag is tightly coupled to current left-unfold ordering; changing that order may hurt search even while all rules remain sound.

Chapter 11

Reflecting executable checks into a certificate

Mathematical intention

A Boolean equation is finite computational evidence. The generic certificate also needs semantic functions: from a false node choose a false successor and world; show that false nodes are live; and prove every followed live edge obeys the propositional witness transition. Reflection proves once that the structure of every accepted FirstOrder graph supplies these fields.

Rocq realization

`FirstOrder/Certification.v` proceeds in four layers.

1. `structural_node_wfb`, `structural_root_bound`, and `structural_root_label` recover facts from `structural_check=true`.
2. `structurally_checked_advance` case-splits a node payload. Open is impossible; a false axiom contradicts its theorem; a rule or backlink invokes the stored `rule_counter_step` and the checked target relation.
3. `false_has_path_of_length` iterates that step, and `structurally_checked_false_live` supplies the exact length used by `liveb`.
4. `trace_transitionb_sound` and `witness_check_transition` reflect the finite witness on each live edge.

The constructor is:

```
Definition certify_raw_check
    (g : Cyclic.graph) (tw : Cyclic.trace_witness)
    (Hcheck : Cyclic.raw_check g tw = true) : Cyclic.certificate.
```

Listing 11.1: The graph-to-certificate bridge.

It fills all semantic fields using the reflection lemmas. Then `raw_check_sound` in `FirstOrder/Certification.v` applies the unchanged generic `certificate_sound`. A one-node axiom graph with an empty witness is packaged by `certify_axiom`; generated logical axioms and theorem-backed benchmark leaves therefore use the same public certificate interface.

Soundness and limitations

The reflection proof does not prove the relational closure algorithm complete. It extracts the structural and witness conjuncts of `raw_check`; the relational conjunct remains an additional executable gate. This is sufficient for soundness because the certificate already contains the concrete routing witness used by the lexicographic descent theorem.

Chapter 12

The concrete definition library and derived mathematics

12.1 Nineteen inductive definitions

Mathematical intention

The definition library is data, not new trusted semantics. Each entry lists Horn clauses in search order. The nineteen names are:

$$N,\ N2,\ E,\ O,\ P,\ Q,\ LIST,\ APP,\ TAKE,\ DROP,$$
$$MINUS,\ LEN,\ PLUS,\ COUNT,\ ADD,\ MUL,\ LEQ,\ R,\ H.$$

They cover naturals and parity, mutually dependent predicates, lists and list relations, arithmetic relations, order, and the hydra-style relation H.

Representative productions are

$$\frac{}{N(0)} \qquad \frac{N(x)}{N(sx)} \qquad \frac{}{E(0)} \qquad \frac{O(x)}{E(sx)} \qquad \frac{E(x)}{O(sx)},$$

and

$$\frac{N(y)}{ADD(0,y,y)} \qquad \frac{ADD(x,y,z)}{ADD(sx,y,sz)}.$$

Rocq realization

`FirstOrder/Definitions.v` defines shorthands `fv`, `ev`, and `clause_of`, then transcribes the source-order clause lists. The final `fo_definitions` association list preserves all nineteen entries. This order matters operationally because lookup returns the first named entry and right/left unfolding enumerate clauses in list order.

For orientation, Table 12.1 summarizes the library.

Table 12.1: Concrete definitions in `FirstOrder/Definitions.v`.

Name	Intended relation and clause pattern
`N`	Unary naturals: zero and successor closure.
`N2`	Binary relation: base $N(y) \Rightarrow N2(0, y)$, then a swapping recursive step.
`E`, `O`	Even/odd mutual recursion through successor.
`P`, `Q`	Mutually supporting unary/binary predicates used by benchmark 05.
`LIST`	Zero as empty list; $LIST(t) \Rightarrow LIST(cons(h, t))$.
`APP`	Relational append over constructor lists.
`TAKE`, `DROP`	Three-place list-prefix and suffix relations with two bases and one recursion.
`MINUS`	Truncated relational subtraction over unary naturals.
`LEN`	Relational list length.
`PLUS`	Standard recursion in the first addition argument.
`COUNT`	Counts equal list elements, with equality by shared syntax and a disequality branch.
`ADD`	Addition variant whose base also requires $N(y)$.
`MUL`	Multiplication transcription; the recursive body names unresolved `ADD_2`.
`LEQ`	Unary less-than-or-equal relation.
`R`	Three clauses used by disabled source benchmarks.
`H`	Six-clause hydra-style relation with three bases and three reductions.

Soundness and limitations

There is no arity checker or definition-file parser. The unresolved `ADD_2` reference inside `MUL` is faithfully present and lookup of that name yields no clauses. This does not corrupt other definitions, but it limits proofs relying on the recursive multiplication clause.

12.2 Introduction, inversion, and rank induction

Mathematical intention

Concrete mathematics uses the two halves of `models`. Introduction lemmas instantiate a clause valuation. Inversion lemmas expose constructor shape and a smaller rank. For example:

$$\mathsf{ranked}(N, x, h) \Rightarrow x = 0 \ \vee \ \exists y, h' < h.\ x = s(y) \wedge \mathsf{ranked}(N, y, h'),$$

and

$$\mathsf{ranked}(O, x, h) \Rightarrow \exists y, h' < h.\ x = s(y) \wedge \mathsf{ranked}(E, y, h').$$

Well-founded induction on h proves both $E(x) \Rightarrow N(x)$ and $O(x) \Rightarrow N(x)$ simultaneously.

Rocq realization

`FirstOrder/Derived.v` defines semantic shorthands `ranked` and `holds`. `model_intro` and `model_elim` project the model record. Introduction lemmas include `n_zero`, `n_succ`, `e_zero`, `e_succ`, `o_`

`succ`, the `P`/`Q` rules, `add_zero/add_succ`, `n2_zero/n2_succ`, and six `H` cases. Inversions are `n_rank_inversion`, `e_rank_inversion`, `o_rank_inversion`, and `add_rank_inversion`.

The mutual theorem `even_odd_natural_by_rank` performs `inductionheightusinglt_wf_ind`; `even_natural` and `odd_natural` hide the existential height. Similar rank inductions prove `natural_parity`, `q_for_naturals`, `add_zero_right`, `add_result_natural`, and `add_shift_right`.

Soundness and limitations

These are direct semantic proofs, independent of graph search. When a public benchmark fallback invokes one of them, the result is a genuine theorem but not evidence that candidate generation found a cyclic graph. That separation is a feature of the current transitional API and must remain visible in reports.

12.3 Rank sums and the hydra measure

Mathematical intention

Some inductions need a combined measure. For $N2(x, y)$, the recursive step swaps arguments, so induction on only one natural rank is inconvenient. `n2_for_n_rank_sum` uses $h_x + h_y$; inversion strictly reduces one component and arithmetic shows the sum falls.

The H relation is more geometric. Define value depth by

$$d(\mathsf{VConst}(n)) = 0, \qquad d(\mathsf{VFun}(f, \vec{v})) = 1 + \max_i d(v_i).$$

For a pair use $\max(d(x), d(y))$. The three recursive shapes satisfy:

$$\begin{aligned} \max(a, b) &< \max(a + 1, b + 2), \\ \max(a + 1, a) &< \max(a + 2, 0), \\ \max(a + 1, a) &< \max(0, a + 2). \end{aligned}$$

These are the step, left-hydra, and right-hydra decreases in the direction needed by backward proof.

Rocq realization

`value_depth` folds `Nat.max` over child depths; `value_depth_succ` gives the unary successor equation. The arithmetic lemmas `max_step_decreases`, `max_left_hydra_decreases`, and `max_right_hydra_decreases` feed `h_for_natural_depth`. The final `h_for_naturals` instantiates the measure with `Nat.max(value_depthx)(value_depthy)`.

Soundness and limitations

The hydra proof is a direct well-founded semantic argument, not a generated cyclic certificate. It establishes benchmark validity and documents the mathematical invariant a future complete search should rediscover. It also shows why "all benchmark truths come from rank of the principal predicate" would be an overstatement: the code uses a separate structural value-depth measure here.

Chapter 13

Benchmarks and the public search contract

13.1 Nine positive judgments

Mathematical intention

The public positive suite contains the enabled source cases 01, 02, 04, 05, 07, 08, 09, 13, and 14. Their formulas are:

Table 13.1: Public FirstOrder benchmark judgments.

Case	Goal	Direct semantic theorem
01	$O(x) \vdash N(x)$	`benchmark_01_valid`
02	$E(x) \lor O(x) \vdash N(x)$	`benchmark_02_valid`
04	$N(x) \vdash O(x) \lor E(x)$	`benchmark_04_valid`
05	$N(x) \land N(y) \vdash Q(x, y)$	`benchmark_05_valid`
07	$N(x) \vdash ADD(x, 0, x)$	`benchmark_07_valid`
08	$N(x) \land N(y) \land ADD(x, y, z) \vdash N(z)$	`benchmark_08_valid`
09	$N(x) \land N(y) \land ADD(x, y, z) \vdash ADD(x, s(y), s(z))$	`benchmark_09_valid`
13	$N(y) \land N(x) \vdash H(x, y)$	`benchmark_13_valid`
14	$N(y) \land N(x) \vdash N2(x, y)$	`benchmark_14_valid`

Rocq realization

`FirstOrder/Benchmarks.v` reifies these judgments with tagged left predicates and untagged right predicates. Each theorem expands `sequent_holds`, extracts the selected antecedent product, removes the free right witness layer with `right_holds_free`, chooses the appropriate right product, and invokes a lemma from `Derived.v`. The inductive type `positive_benchmark` and `benchmark_judgment` provide an exact finite index. `prove_positive_benchmark` returns an `axiom_instance` whose theorem field is the corresponding kernel-checked validity proof.

Soundness and limitations

The six disabled source cases 03, 06, 10, 11, 12, and 15 are not public negative theorems and are not established invalid. They are also not all persisted as bounded non-success tests. Absence from the positive list should not be read semantically.

13.2 Verified proofs and iterative deepening

Mathematical intention

The user-facing result should not expose an arbitrary theorem callback. It should contain a certificate, an equality connecting its normalized root to the user's goal, and a proof that the certificate checker passed. Validity is then derived uniformly.

Rocq realization

```
Record verified_proof (goal : judgment) : Type := {
  proof_certificate : Cyclic.certificate;
  proof_root_matches :
    proof_certificate.(Cyclic.cert_graph).(Cyclic.graph_root) =
      normalize_judgment goal;
  proof_check_passes :
    Cyclic.check_certificate proof_certificate = true
}.

Theorem search_sound config goal :
  solvedb config goal = true -> Cyclic.Valid goal.
```

Listing 13.1: The public proof-carrying result and soundness contract.

`verified_proof_valid` applies `certificate_sound`, rewrites the root equality, and transports across `normalize_judgment_valid`. `search_depths` tries successive depths from `min_depth` through `max_depth`; `default_config` is 1 through 11.

At each depth, `search_at_depth` tries:

1. a generated logical axiom;
2. general certification-aware graph search;
3. nine exact normalized benchmark matches, guarded by conservative depth thresholds and packaged by `theorem_strategy`.

Soundness and limitations

The fallback dispatcher is transitional but sound. It compares the normalized input exactly to a benchmark and turns an existing validity theorem into a one-node checked axiom certificate. Consequently the public statement remains Equation (1.1); the limitation is only what the result says about general proof discovery.

13.3 Reproduced public computations

At the documented revision, a clean Rocq evaluation produced:

```
Eval vm_compute in map (solvedb default_config) positive_goals.
(* = [true; true; true; true; true; true; true; true; true] *)
```

Listing 13.2: The nine public outcomes, reproduced with vm_compute.

The source also fixes two important controls. At depth one, `benchmark_01` returns `false`; this means only bounded non-success. An arbitrary reflexive identity outside the benchmark table returns `true`, while an unsupported false-looking empty-to-empty judgment is not claimed.

> **Reading invariant.** The nine Booleans test the public API, including exact theorem fallbacks. The 15-node computation in Chapter 14 separately tests general graph search for benchmark 01.

Chapter 14

Complete vertical slice: $O(x) \vdash N(x)$

14.1 The direct mathematical proof

Mathematical intention

The parity definitions are

$$\frac{}{E(0)} \qquad \frac{O(x)}{E(sx)} \qquad \frac{E(x)}{O(sx)}.$$

Naturals are generated by zero and successor. The desired theorem is $O(x) \Rightarrow N(x)$. Because $O(x)$ has a finite derivation rank, invert it to $x = s(y)$ and a strictly smaller ranked proof of $E(y)$. Mutual well-founded induction proves the stronger pair

$$\forall x.\ E(x)@h \Rightarrow N(x), \qquad \forall x.\ O(x)@h \Rightarrow N(x).$$

In the odd case, the smaller even premise yields $N(y)$, then the natural successor clause yields $N(sy) = N(x)$.

Rocq realization

This is `even_odd_natural_by_rank` in `FirstOrder/Derived.v`, followed by `odd_natural` in `FirstOrder/Derived.v`. The benchmark theorem merely unwraps sequent semantics and calls `odd_natural`:

```
Theorem benchmark_01_valid : Cyclic.Valid benchmark_01.
Proof.
  intros [i rho]. unfold benchmark_01, denote, sequent_holds; simpl.
  intros Hmodel [p [[Hp_eq | []] Hp]]. subst p. simpl in Hp.
  finish_free_right. exists [pred "N" [x]]. split; [now left|].
  simpl. split; [|exact I]. now apply odd_natural.
Qed.
```

Listing 14.1: The semantic endpoint used by the benchmark theorem.

Soundness and limitations

This direct proof is independent evidence of validity. It does not mention a proof graph, backlinks, or the global trace checker. The next section shows that general search can also construct a cyclic certificate for the same goal at a larger explicit fuel.

14.2 The verified 15-node cyclic graph

Mathematical intention

Cyclically, unfold oddness once, use its equation to expose a successor, unfold the natural goal, then recursively reason from evenness. Unfolding evenness splits into a zero base branch and a successor recursive branch. The base branch closes by identity after the natural base clause. The recursive branch returns to the original odd-to-natural shape after weakening and renaming. Strict progress occurs on the left-unfold edges, so the final identity backlink may itself be non-progressing.

Rocq realization

The following computations were reproduced in Rocq at the documented revision:

```
Eval vm_compute in
  match certify_searched_graph 16 (normalize_judgment benchmark_01) with
  | Some (existT _ g _) => List.length g.(Cyclic.graph_nodes)
  | None => 0
  end.
(* = 15 *)

(* Node labels in index order: *)
["L.Unfold"; "Eq subst"; "Simpl ="; "R.Unfold/match";
 "L.Unfold"; "Eq subst"; "Eq subst"; "Simpl ="; "Simpl =";
 "R.Unfold/match"; "R.Unfold/match"; "Id/modulo-tags";
 "Weaken"; "Subst"; "Backlink"]

(* Outgoing target indices in the same order: *)
[[1]; [2]; [3]; [4]; [5;6]; [7]; [8]; [9]; [10]; [11];
 [12]; []; [13]; [14]; [0]]
```

Listing 14.2: General graph-search evidence for benchmark 01.

Figure 14.1 renders exactly that adjacency. The branch at node 4 is the even base/recursive split. Node 11 is the only axiom leaf. Node 14 links to the root.

For readability, Table 14.1 abstracts the exact terms while preserving the logical role of every node.

Table 14.1: Reading the 15 nodes of the benchmark-01 graph.

Index	Payload	Mathematical effect
0	L.Unfold	Invert tagged $O(x)$; introduce $E(y)$, contraction $O(x)$, and $x = s(y)$. The recursive premise rank strictly decreases.
1	Eq subst	Replace x by $s(y)$ throughout the branch.
2	Simpl =	Remove the resulting reflexive successor equation.
3	R.Unfold/match	Match the successor clause of $N(s(y))$, reducing the goal to $N(y)$.
4	L.Unfold	Invert $E(y)$, producing zero and successor branches.
5	Eq subst	Base branch: replace y by zero.
6	Eq subst	Recursive branch: replace y by a successor.
7	Simpl =	Base branch removes its reflexive equality.

Index	Payload	Mathematical effect
8	Simpl =	Recursive branch removes a reflexive constructor equality.
9	R.Unfold/match	Base branch unfolds $N(0)$ to the empty product.
10	R.Unfold/match	Recursive branch unfolds the new natural-successor goal.
11	Id/modulo-tags	Base branch closes because the empty RHS product is true.
12	Weaken	Recursive branch drops context not required by the companion.
13	Subst	Rename the remaining free variable to match the root exactly.
14	Backlink	Close the recursive branch against node 0.

The machine order and the diagram now coincide: node 11 is the base axiom, node 12 begins recursive-branch preparation, and node 14 links to node 0.

Soundness and limitations

The graph is accepted not because it has a backlink, but because all local records are checked and the global witness passes. The reproduced witness selected tags

$$[0, 0, 0, 0, 0, \bot, 0, \bot, 0, \bot, 0, \bot, 0, 0, 0]$$

and potentials

$$[0, 3, 2, 1, 0, 0, 6, 0, 5, 0, 4, 0, 3, 2, 1].$$

Here $\bot$ denotes `None`. Progress is carried on left-unfold paths; potentials bridge the intervening non-progressing edges. The cycle closes only after the strict semantic decrease exposed at node 0 (and again along the recursive even branch).

14.3 Cyclic proof and direct induction are the same descent seen twice

The direct theorem descends on the hidden clause derivation height. The cyclic proof exposes the same descent locally on tagged left-unfold edges and lets a finite graph represent repetition. `odd_natural` packages the induction globally in a Rocq proof term; the 15-node certificate packages it as a regular derivation plus a trace witness. Neither is more semantically privileged: both ultimately rely on the ranked inversion condition of `models`.

The comparison is useful when extending search. A direct lemma identifies a mathematical invariant and supplies an independent oracle for a benchmark. A generated graph tests whether the operational calculus, ordering, matching, backlink preparation, and witness construction can discover that invariant. Conflating these tests would hide exactly where automation remains incomplete.

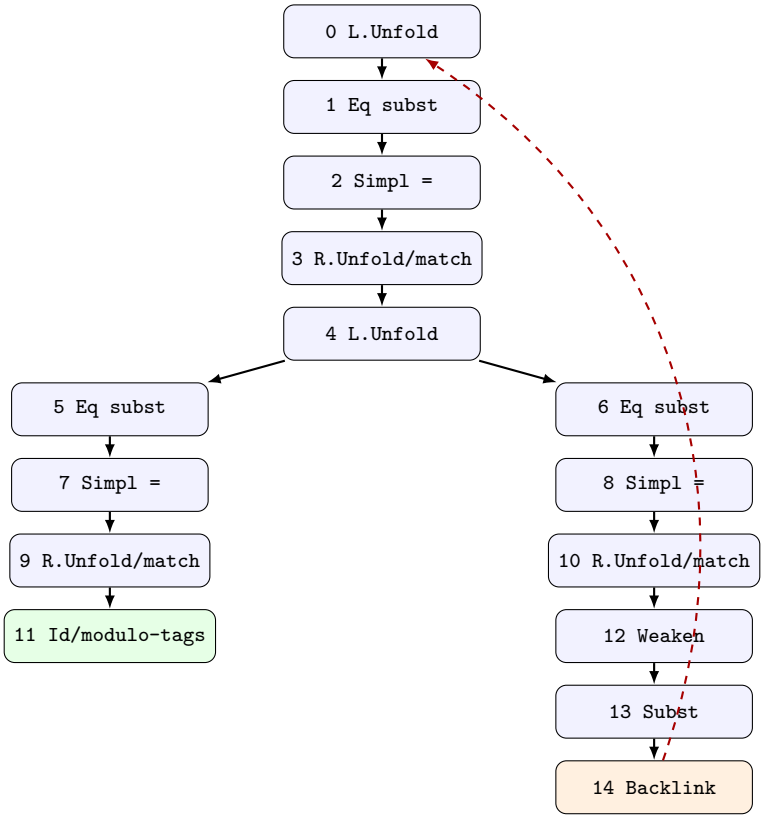


Figure 14.1: The certified benchmark-01 graph. The labels and targets are reproduced by computation; the layout merely separates the base and recursive branches.

Chapter 15

Limitations and precise claims

15.1 What is implemented soundly

The reusable core consists of reified DNF syntax, ranked free-constructor semantics, normalization, theorem-backed rule records, graph construction, structural and trace checking, Boolean reflection into a semantic certificate, and the one-way public theorem. Identity, ex falso, propositional splitting, the listed equality simplifiers, explicit weakening/substitution, left and right unfolding, and exact backlinks all have local semantic and countermodel proofs.

The strongest safe summary is:

> If `solvedbcJ` computes to `true`, then the Rocq kernel can derive `Cyclic.ValidJ`. Search bugs may cause missed proofs, expensive reduction, or a rejected graph; they cannot bypass rule proofs or the certificate soundness theorem.

15.2 Missing rule families

Folding. There is no folding generator and no fold-then-backlink phase. This is one of the largest differences from the source prover and prevents useful companions from being synthesized in the reverse direction of a definition.

General existential instantiation. There is no rule that tries available universal/free variables as instantiations for an arbitrary sequent existential. The right equation solver is much narrower. A new rule must be proved compatible with the base/witness split; naïve whole-sequent substitution is not automatically sound under the current semantics.

Exact multi-unification for right unfolding. The verified symmetric term unifier exists, but right unfolding uses directional matching or an equation-producing fallback. It does not reproduce the source prover's multi-unification and residual-equation behavior.

Composed left/right unfolding. The original priority includes a composed step. Here the two rules may occur on consecutive nodes, but there is no dedicated composed candidate or its search scheduling.

15.3 Restricted implemented families

- Right conjunction requires fully free left and right pieces rather than pairwise-disjoint existential sets.
- Left equality substitution recognizes only a free variable on the left and a free replacement.
- Right existential solving accepts a single existential variable equated to a free term; harder cyclic or mixed equations are rejected.
- Right discharge requires a singleton antecedent and a free discharged atom.
- Left unfolding requires a singleton antecedent, a tagged selected atom, bounded variables, and a free RHS.
- Prepared backlink matching is directional and exact after substitution; weakening removes only a suffix beyond a matching prefix.
- The final backlink is exact apart from predicate tags and cannot reorder set-like products or formulas.

Each restriction is conservative. Removing one requires a theorem, not only a new Boolean branch.

15.4 Search and interface limitations

Search is bounded and its fuel counts graph expansions. `None` means "not found within this scheduling and bound." It can result from missing rules, restrictive side conditions, candidate order, a poor backlink, an invalid closed graph, witness-enumeration cost, or too little fuel. There is no wall-clock timeout inside Gallina.

The public nine-result test includes the benchmark dispatcher. General graph search has reproduced benchmark 01 at fuel 16 and, according to the maintained audit, benchmark 04 at fuel 18; these larger computations are not default-depth coverage of the full suite. The dispatcher should be removed only after persisted general-search certificates cover each intended benchmark.

There is no parser for the source textual language, no definition-file loader, no pretty-printer compatible with the CLI, no command-line proof command, no timeout integration, and no proof-statistics interface. Users construct reified judgments in Gallina.

15.5 Soundness is not completeness

The implementation proves

$$\mathsf{success} \Rightarrow \mathsf{validity}.$$

It does not prove

$$\mathsf{validity} \Rightarrow \mathsf{success}, \qquad \mathsf{failure} \Rightarrow \mathsf{invalidity}, \qquad \text{or} \qquad \mathsf{success} \Rightarrow \mathsf{generalDFSFoundIt}.$$

The last implication fails specifically because a theorem-backed exact benchmark fallback is a permitted public strategy. This logical bookkeeping is the most important interpretive limitation in the current system.

Chapter 16

Trust boundary and reproduced validation

16.1 What the kernel checks

Every `Qed` or `Defined` object is checked by the Rocq kernel. Rule records contain proof terms; Boolean reflection lemmas are proof terms; the certificate fields are proof terms; and the final well-founded induction is a proof term. Tactics such as `lia` and evaluation mechanisms such as `vm_compute` help construct or reduce those terms but do not add unchecked object-logic rules. The Rocq reference manual describes this kernel-centered trust model and the relevant commands [5].

The trusted computing base still includes Rocq's kernel and runtime, the standard library definitions and admitted principles explicitly reported by `PrintAssumptions`, the parser/elaborator used to construct terms, and the host/compiler stack needed to run Rocq. The LaTeX tutorial is explanatory and not part of the proof object.

16.2 Printed assumptions

The reproduced queries at the documented revision gave:

```
Print Assumptions benchmark_01_valid.
(* Closed under the global context *)

Print Assumptions raw_check_sound.
(* Classical_Prop.classic *)

Print Assumptions Cyclic.certificate_sound.
(* Classical_Prop.classic *)
```

Listing 16.1: Assumption audit.

For `search_sound`, Rocq reported the following four standard-library principles:

- `FunctionalExtensionality.functional_extensionality_dep`;
- `ClassicalEpsilon.constructive_indefinite_description`;
- `Description.constructive_definite_description`;
- `Classical_Prop.classic`.

The direct benchmark theorem is constructive with respect to the global context, whereas the generic cyclic route uses classical reasoning to turn a double-negated validity argument into validity. Search and normalization bring in functional extensionality and choice/description principles through their dependencies.

Soundness and limitations

"No project-specific axioms" is more accurate than "axiom-free." The printed output is the right evidence. It should be rerun after changes because refactoring definitions can alter the transitive assumption set even when a theorem statement remains unchanged.

16.3 Build and computation record

At revision `08f6cd8834a3fc22088a064025abb9ee65c7f545`, before tutorial generation, the command

```
make -C Rocq-copy -B -j2
```

completed successfully. It compiled the Cyclic framework, the entire FirstOrder stack, and the other configured developments. During that build, assumption-printing targets reported classical logic and, for neighboring modules, constructive definite description as expected.

The following results were independently reproduced in a Rocq 9.1.0 REPL against the compiled tree:

- the public Boolean vector is nine `true` values;
- `certify_searched_graph16` on normalized benchmark 01 returns a 15-node checked graph;
- its exact node labels and target lists are those in Chapter 14;
- its witness tags and potentials are those recorded there;
- the assumption outputs are those listed above.

The project build is rerun after document generation as a regression guard.

Chapter 17

A theorem-first extension checklist

Adding a search feature safely is easiest when the theorem boundary is fixed before the generator. The following sequence scales from a small simplifier to folding or general backlink subsumption.

1. **State the mathematical inference.** Give source and premise sequents, witness behavior, and every freshness, freeness, arity, and occurrence side condition.
2. **Prove forward validity.** Assume all premises valid and derive the conclusion at an arbitrary world. Do not mention search order.
3. **Prove countermodel propagation.** From a false conclusion choose an actual premise index, a target world, and a false premise.
4. **Design trace edges.** List every valid tag pair and the subset that strictly progresses. Prove the non-increase and decrease inequalities.
5. **Package a `rule_instance`.** Only now select the human-readable rule name and assemble both proof fields.
6. **Write a conservative recognizer.** Turn executable side-condition successes into proof inputs through reflection lemmas or dependent pattern matching.
7. **Recheck computed artifacts.** For matching or unification, apply the result and compare the entire structured output, not only flattened terms.
8. **Choose candidate order explicitly.** Document interactions with simplification, unfolding, backlinks, DFS fuel, and trace-tag selection.
9. **Add positive and negative regression equations.** Cover success, near misses, arity errors, variable kinds, occurs checks, and list positions.
10. **Add a checked graph regression.** Demonstrate that a nontrivial result reaches `raw_check=true`, not merely `graph_closedb=true`.
11. **Run the public soundness path.** Produce a `verified_proof` and derive `Cyclic.Valid`; never create a second unchecked result type.
12. **Audit assumptions and bounded failures.** Run `PrintAssumptions`; label every `None` result with its exact bound and avoid semantic conclusions from it.

For the major missing families, the first proof obligations are already clear. Folding needs semantic equivalence or a countermodel-directed rule plus trace edges. Existential instantiation needs a theorem that respects the separation of base and witness valuations. Full right multi-unification needs a verified result containing a substitution and residual equations. General backlinks need set-level sequent subsumption and an explicit path from the current node to an exact companion.

Appendix A

Mathematics-to-Rocq correspondence

Table A.1: Correspondence between mathematical objects and checked symbols.

Mathematical notion	Primary Rocq symbol	Important qualification
Two variable classes	`var_kind`, `variable`	Same numeric ID may occur in each distinct kind.
Free term algebra	`term`; `value`; `eval_term`	Semantic equality is constructor-tree equality.
Tagged predicate atom	`Pred:optionnat->string->listterm->atom`	Tag affects traces, never truth.
Conjunction	`product=listatom`; `product_holds`	Empty product is true.
DNF disjunction	`formula=listproduct`; `formula_holds`	Empty formula is false.
Sequent and environment	`sequent`; `judgment`	Judgment stores definitions with the sequent.
Horn production	`clause_body`; `clause_head`	Predicate name supplied by definition lookup.
Finite substitution	`substitution`; `subst_term`	First binding wins.
Right existential witness	`merge_valuation`; `right_holds`	One witness is shared by a chosen product.
Ranked model	`interpretation`; `models`	Introduction plus decreasing inversion.
Trace rank	`rank_atom`; `rank_formula`; `rank_of_tag`	First matching antecedent tag; missing is zero.
Local rule soundness	`rule_theorem`	Premise validity implies conclusion validity.
Countermodel step	`rule_counter_step`	Produces premise index, false world, and edge respect.

Mathematical notion	Primary Rocq symbol	Important qualification
Non-increase/progress	`edge_valid`; `edge_progress`; `respects_edge`	Progress list is checked as a subset of valid pairs.
Finite regular proof	`graph`; `node_kind`	Backlinks target stored nodes.
Structural validity	`structural_check`	Includes reachability and absence of open nodes.
Size-change closure	`relational_trace_check`; `relational_check`	Required acceptance gate.
Finite trace routing	`trace_witness`; `witness_check`	Concrete witness used in semantic descent.
Proof certificate	`certificate`; `certify_raw_check`	Adds semantic evidence not decidable by generic Boolean code.
Infinite descent	`descent_measure`; `certificate_step_decreases`	Two-phase nested lexicographic order.
Semantic validity	`Cyclic.Valid`; `certificate_sound`	Universal over semantic worlds.
Set-style normalization	`normalize_judgment`	Removes duplicates but does not sort.
Directional match	`match_terms`; `match_terms_sound`	Pattern-to-target only; result rechecked.
Symmetric unification	`unify_terms`; `unify_terms_sound`	Occurs checked and fuel bounded; not fully integrated.
Candidate generation	`generate_axiom`; `simplification_candidate`; `generate_rules`	Order affects bounded discovery only.
Checked graph search	`certify_searched_graph`	Rejects invalid closed graphs and continues DFS.
Public decision bit	`solvedb`	One-way success contract only.
Public theorem	`search_sound`	Includes transitional benchmark strategies.

Appendix B

Module and symbol index

Table B.1: All documented modules and their principal symbols.

Module	Principal symbols and role
`FirstOrder/Syntax.v`	`var_kind`, `variable`, `term`, `atom`, `product`, `formula`, `sequent`, `clause`, `judgment`, exact/tag-insensitive equalities, substitutions, shifts, bounds, tags, preparation, and definition lookup.
`FirstOrder/Semantics.v`	`value`, `valuation`, `interpretation`, `atom_holds`, `product_holds`, `formula_holds`, `merge_valuation`, `right_holds`, `models`, `denote`, ranks, free-fragment lemmas, and shifting lemmas; instantiates `Framework.Make`.
`FirstOrder/Automation.v`	Duplicate normalization with semantic preservation; set-style Booleans; variable/freshness analysis; `match_terms`; substitution composition; occurs checking; `unify_terms`.
`FirstOrder/Rules.v`	The semantic kernels for all axioms and rules: identity/ex-falso, propositional rules, equality rules, discharge, weakening, substitution, right/left unfolding, and backlinks; includes arbitrary-position transport.
`FirstOrder/RuleGeneration.v`	Ordered scans for axioms and simplification; structural candidates; matched and standard right unfold; arbitrary-position left unfold; `generate_rules`; executable regressions.
`FirstOrder/GraphSearch.v`	First-open expansion, exact and prepared backlinks, bounded DFS, witness heuristics/enumeration, `checked_graph`, certification-aware DFS, and `certify_searched_graph`.
`FirstOrder/Certification.v`	Reflection of structure and witnesses, countermodel advance, liveness, `certify_raw_check`, `raw_check_sound`, and one-node axiom certificates.
`FirstOrder/Definitions.v`	AST transcription of all 19 entries in `fo_definitions`, retaining source clause order.

Module	Principal symbols and role
`FirstOrder/Derived.v`	Clause introduction/inversion lemmas, parity/natural rank induction, `P`/`Q` results, addition results, rank sums for `N2`, value depth, and the hydra proof.
`FirstOrder/Benchmarks.v`	Nine positive judgments, direct validity theorems, finite benchmark index, and proof-producing exact dispatcher.
`FirstOrder/Search.v`	`search_config`, `verified_proof`, normalized root transport, generated and theorem strategies, iterative deepening, `search`, `solvedb`, and `search_sound`.
`Cyclic/Relations.v`	Two-point slope algebra, relation/path composition, bounded closure, and decreasing-diagonal checks.
`Cyclic/Framework.v`	Object-logic interface, rule/axiom records, finite graph operations, structural/relation/witness checks, certificate, lexicographic measure, and `certificate_sound`.

Appendix C

Rule quick reference

This appendix collects the mathematical forms in one place. Side conditions are abbreviated; Chapters 5–7 give the semantic reason for each one.

$$\frac{B \subseteq_\tau A}{A \vdash R_0 \vee B \vee R_1}\ \mathrm{Id}/\tau \qquad \frac{}{A, t \neq t, A' \vdash R}\ \bot_L \qquad \frac{}{A, c = f(\vec{t}), A' \vdash R}\ \mathsf{clash}_L$$

$$\frac{p \vdash R \quad P \vdash R}{p \vee P \vdash R}\ L.\vee \qquad \frac{L \vdash a \quad L \vdash B}{L \vdash a \wedge B}\ R.\wedge$$

$$\frac{A, A' \vdash R}{A, t = t, A' \vdash R}\ \mathsf{Simpl}{=}L \qquad \frac{L \vdash R_0 \vee (B \wedge B') \vee R_1}{L \vdash R_0 \vee (B \wedge t = t \wedge B') \vee R_1}\ \mathsf{Simpl}{=}R$$

$$\frac{A[x := t], A'[x := t] \vdash R[x := t]}{A, x = t, A' \vdash R}\ \mathsf{EqSubst} \qquad \frac{L \vdash R_0 \vee (B[\mathsf{E}_x := t] \wedge B'[\mathsf{E}_x := t]) \vee R_1}{L \vdash R_0 \vee (B \wedge \mathsf{E}_x = t \wedge B') \vee R_1}\ \exists\mathsf{Solve}$$

$$\frac{a, A \vdash R_0 \vee (B \wedge B') \vee R_1}{a, A \vdash R_0 \vee (B \wedge a \wedge B') \vee R_1}\ \mathsf{Discharge}_R$$

$$\frac{A, \bigwedge_i s_i = t_i, A' \vdash R}{A, f(\vec{s}) = f(\vec{t}), A' \vdash R}\ \mathsf{Inj}_L \qquad \frac{L \vdash R_0 \vee (B \wedge \bigwedge_i s_i = t_i \wedge B') \vee R_1}{L \vdash R_0 \vee (B \wedge f(\vec{s}) = f(\vec{t}) \wedge B') \vee R_1}\ \mathsf{FunEq}_R$$

$$\frac{A \vdash R}{A, E \vdash R}\ \mathsf{Weak} \qquad \frac{L \vdash R}{\sigma(L) \vdash \sigma(R)}\ \mathsf{Subst} \qquad \frac{J}{J}\ \mathsf{Backlink}$$

$$\frac{L \vdash R_0 \vee (A \wedge \sigma(B) \wedge A') \vee R_1}{L \vdash R_0 \vee (A \wedge P(\vec{t}) \wedge A') \vee R_1}\ R.\mathsf{Unfold/match} \quad (\sigma\vec{h} = \vec{t})$$

$$\frac{L \vdash R_0 \vee (A \wedge B^{\exists} \wedge \vec{t} = \vec{h}^{\exists} \wedge A') \vee R_1}{L \vdash R_0 \vee (A \wedge P(\vec{t}) \wedge A') \vee R_1}\ R.\mathsf{Unfold}$$

$$\frac{\{A, B_c^{\uparrow, t_c}, \vec{u} = \vec{h}_c^{\uparrow}, P^t(\vec{u}), A' \vdash R\}_{c \in \mathsf{def}(P)}}{A, P^t(\vec{u}), A' \vdash R}\ L.\mathsf{Unfold}.$$

Appendix D

End-to-end audit recipe

When evaluating a claimed proof from this development, the following compact procedure reconstructs the entire theorem path.

1. Write the exact judgment: definition environment, DNF antecedent and succedent, variable kinds, and tags.
2. Normalize it and record which duplicate atoms/products disappear.
3. Determine whether `generate_axiom` succeeds. If not, record the first simplifier, backlink candidates, and `generate_rules` list in their actual order.
4. For every installed rule inspect its `rule_theorem`, `rule_counter_step`, and edge information. For an axiom inspect `axiom_theorem`.
5. Distinguish an explicit substitution/weakening preparation node from the final exact backlink.
6. Verify all graph nodes are reachable and closed; then separately run the relational and finite-witness checks.
7. Record the concrete witness tags and potentials. Identify where the cycle makes semantic progress and where auxiliary potential bridges stay edges.
8. Follow `certify_raw_check`: structure supplies root, liveness, and advance; witness reflection supplies trace transitions.
9. Apply `certificate_sound`, rewrite the normalized root equality, and use normalization preservation to obtain validity of the input.
10. If the public API was used, determine whether the result came from a generated axiom, general graph search, or an exact benchmark theorem strategy.
11. If computation returns `None` or `false`, record the exact configuration and report only bounded non-success.
12. Run `PrintAssumptions` on the exported theorem and preserve the output with the result.

For benchmark 01 this recipe yields the chain

$$
\begin{aligned}
O(x) \vdash N(x) &\xrightarrow{L.\mathsf{Unfold}} \text{ranked parity premises} \\
&\xrightarrow{\mathsf{EqSubst},\mathsf{Simpl}{=},R.\mathsf{Unfold}} \text{even base/recursive split} \\
&\xrightarrow{\text{base}} \mathrm{Id} \qquad \text{or} \qquad \xrightarrow{\text{recursive}} \mathsf{Weak};\mathsf{Subst};\mathsf{Backlink} \\
&\xrightarrow{\mathsf{raw_check{=}true}} \mathsf{certificate} \xrightarrow{\mathsf{certificate_sound}} \mathsf{Valid}(O(x) \vdash N(x)).
\end{aligned}
$$

Conclusion

The FirstOrder development turns cyclic proof search into proof-producing automation by keeping three layers separate. The mathematical layer gives a free-constructor domain, shared right witnesses, and ranked inductive definitions. The local proof layer packages every inference with both forward validity and countermodel propagation. The global layer checks a finite graph, reflects a concrete trace witness, and rules out a countermodel by lexicographic infinite descent.

The implementation's limitations are substantial but cleanly fenced. Folding and general existential instantiation are missing; matching, conjunction, left unfolding, weakening, and backlinks are restricted; fuel makes failure only bounded non-success; and the public benchmark suite still includes exact theorem-backed fallbacks. There is no parser or CLI. None of those gaps weakens the proved success direction. They define the agenda for theorem-first extension: prove the semantic transformation and trace behavior, package it as a rule record, expose it through a conservative generator, and reuse the same checked graph-to-validity path.

Bibliography

[1] James Brotherston. Cyclic proofs for first-order logic with inductive definitions. In Bernhard Beckert, editor, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 3702 of *Lecture Notes in Computer Science*, pages 78–92. Springer, 2005. doi:10.1007/11554554_8.

[2] James Brotherston and Alex Simpson. Sequent calculi for induction and infinite descent. *Journal of Logic and Computation*, 21(6):1177–1216, 2011. doi:10.1093/logcom/exq052.

[3] James Brotherston, Nikos Gorogiannis, and Rasmus Lerchedahl Petersen. A generic cyclic theorem prover. In Ranjit Jhala and Atsushi Igarashi, editors, *Programming Languages and Systems: 10th Asian Symposium, APLAS 2012*, volume 7705 of *Lecture Notes in Computer Science*, pages 350–367. Springer, 2012. doi:10.1007/978-3-642-35182-2_25.

[4] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. The size-change principle for program termination. In *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 81–92. Association for Computing Machinery, 2001. doi:10.1145/360204.360210.

[5] The Rocq Development Team. *The Rocq Prover Reference Manual, Version 9.1.* The Rocq Prover Project, 2026. `https://rocq-prover.org/doc/v9.1/refman/index.html`.